

Nim

Ошибки, модули и коллекции

Как организовать код и работать с данными

Житков Никита · Сагун Александр · Жуков Александр

Исключения / Ввод-вывод / Nimble / Generics / Коллекции

Исключения: обнаружить и обработать

p01_exceptions.nim · СОКРАЩЁННЫЙ ФРАГМЕНТ

```
proc safeDivide(a, b: int): float =  
  if b == 0:  
    raise newException(ValueError,  
      "деление на ноль")  
  result = a.float / b.float  
  
try:  
  echo safeDivide(10, 0)  
except ValueError as e:  
  echo "Поймано ValueError: ", e.msg  
finally:  
  echo "Блок finally выполнен"
```

В примере ValueError создаём явно. Это правило safeDivide, а не поведение оператора /.

CatchableError

Ожидаемые ошибки, которые программа может обработать. ValueError — один из примеров.

Defect

Нарушение правил программы, например выход за границы. Обычно исправляют причину.

finally выполняется при обычном выходе и раскрутке стека исключения.

Проверки: assert и doAssert

p02_assert.nim · ВСЕ ЧЕТЫРЕ ПРОВЕРКИ ПРОХОДЯТ

```
assert 1 + 1 == 2
doAssert 2 * 2 == 4, "Ошибка"

doAssertRaises(ValueError):
  raise newException(ValueError, "тест")

doAssertRaises(IndexDefect):
  let s = @[1, 2, 3]
  discard s[10]
```

assert

Отключается через
--assertions:off.

doAssert

Сохраняет проверку независимо от
--assertions:off.

-d:release включает оптимизации и сохраняет проверки.

-d:danger отключает проверки безопасности: тест s[10] так запускать нельзя.

Ввод-вывод: консоль и файлы

p03_io.nim · ОСНОВНЫЕ ОПЕРАЦИИ

```
import std/os

echo "Это вывод через echo."
stdout.write("Без автоматического перевода")

let path = "example_output.txt"
writeFile(path,
  "Привет из Nim!\nСтрока 2\nСтрока 3\n")
let content = readFile(path)
echo content
for line in lines(path):
  echo "  > ", line
```

echo / write

echo добавляет перевод строки. write выводит переданные байты.

readFile / lines

Прочитать файл целиком или последовательно по строкам.

stdin.readLine()

Чтение строки из консоли. В этом проекте ввод не запрашивается.

Что показывает файловый пример

p03_io.nim · ПОСЛЕ ЗАПИСИ ФАЙЛА

```
echo getFileSize(path)
echo fileExists(path)
removeFile(path)
echo fileExists(path)
```

СОДЕРЖИМОЕ ФАЙЛА

Привет из Nim!

Строка 2

Строка 3

53 байта в UTF-8

30 видимых символов + 3 перевода строки.
Кириллица здесь занимает по два байта.

РЕЗУЛЬТАТЫ ИЗ ЛОГА

Размер: 53

До удаления: true

После: false

В исходном stdout.write строка заканчивается на \n: перевод задан явно в аргументе.

Модули и видимость имён

p04_modules_util.nim

```
proc square*(x: int): int =  
  x * x  
  
proc cube(x: int): int =  
  x * x * x
```

p05_modules_usage.nim · ФРАГМЕНТ

```
import p04_modules_util  
  
echo square(5)  
echo p04_modules_util.square(9)  
# cube(3) недоступна снаружи
```

***** — экспорт

Без звёздочки объявление остаётся внутри модуля.

import

Подключает модуль. `from ... import ...` выбирает имена.

module.name

Квалифицированное имя уточняет, из какого модуля символ.

Пакеты и менеджер Nimble

ПРИМЕР lesson02.nimble · НЕ ВХОДИТ В АРХИВ

```
version = "0.1.0"
author = "Команда 15"
description = "Учебные примеры Nim"
license = "MIT"
srcDir = "src"
bin = @["main"]
requires "nim >= 2.0.0"
```

`nimble install <имя_пакета>` — установка пакета.

Для нашего проекта достаточно Nim: все импорты — стандартная библиотека.

Модуль и пакет

Модули объединяются в пакет с описанием, зависимостями и задачами сборки.

В КАТАЛОГЕ ПАКЕТА

```
nimble init
nimble build
nimble run
```

Generics: один алгоритм, разные типы

p06_generics.nim · ОБЪЯВЛЕНИЕ И ИСПОЛЬЗОВАНИЕ

```
proc myMax[T](a, b: T): T =  
  if a > b: a else: b  
  
type Pair[A, B] = object  
  first: A  
  second: B  
  
let p = Pair[int, string](  
  first: 42, second: "answer")
```

ВЫВОД ПРОЕКТА

```
myMax(3, 7)          = 7  
myMax(3.14, 2.71) = 3.14  
myMax("a", "b")     = b
```

```
p.first = 42  
p.second = answer
```

T выводится из аргументов. Для Pair два типа указаны явно.
myMax подходит типам, для которых определён оператор >.

Свой Stack[T]: добавление и извлечение

p06_generics.nim · ИСПРАВЛЕННЫЙ ФРАГМЕНТ

```
type Stack[T] = object
  items: seq[T]

proc push[T](s: var Stack[T]; item: T) =
  s.items.add(item)

proc pop[T](s: var Stack[T]): T =
  if s.items.len == 0:
    raise newException(ValueError, "стек пуст")
  result = s.items[^1]
  s.items.setLen(s.items.len - 1)
```

LIFO

Последним добавлен — первым извлечён.

ИЗ ПРОЕКТА

Stack[int] → 30
Stack[string] → rocks

Добавлена проверка пустоты. var позволяет изменить сам стек.

array и seq: фиксированный и растущий

p07_collections.nim · ARRAY

```
var arr: array[3, int] = [10, 20, 30]
echo arr[1] # 20
echo arr.len # 3
```

p07_collections.nim · SEQ

```
var s = @[1, 2, 3]
s.add(4)
s.add(5)
echo s # @[1, 2, 3, 4, 5]
```

Семантика значения

Обычное присваивание `seq[int]` создаёт независимое значение.

Элементы типа `ref` могут ссылаться на общие объекты.

У `array` размер входит в тип. У `seq` длина меняется во время выполнения.

map и filter: преобразовать и отобрать

p07_collections.nim · ФРАГМЕНТ

```
import std/sequtils
let s = @[1, 2, 3, 4, 5]

let doubled = s.map(proc(x: int): int = x * 2)
let evens = s.filter(proc(x: int): bool = x mod 2 == 0)
```

MAP — НОВОЕ ЗНАЧЕНИЕ ДЛЯ КАЖДОГО ЭЛЕМЕНТА

@[2, 4, 6, 8, 10]

FILTER — ТОЛЬКО ЧЁТНЫЕ ЭЛЕМЕНТЫ

@[2, 4]

Обе операции возвращают новые последовательности. Исходная `s` остаётся прежней.

set и HashSet: уникальные элементы

ВСТРОЕННЫЙ SET

```
var st: set[char] = {'a', 'b', 'c'}
incl(st, 'd')
excl(st, 'a')
echo 'b' in st    # true
echo card(st)    # 3
```

set[char]: {'b', 'c', 'd'}

Небольшой диапазон порядкового типа.

Например, char или enum.

HASHSET ИЗ СТАНДАРТНОЙ БИБЛИОТЕКИ

```
import std/sets
var langs = initHashSet[string]()
for name in ["Nim", "Python",
             "Rust", "Nim"]:
    langs.incl(name)
echo langs.len    # 3
```

HashSet[string]: три языка

Для элементов нужны hash и ==.

Порядок обхода не гарантирован.

Table: значения по ключам

```
p08_collections_advanced.nim · TABLE[K, V]
```

```
import std/tables
```

```
var ages = initTable[string, int]()
```

```
ages["Alice"] = 30
```

```
ages["Bob"] = 25
```

```
ages["Charlie"] = 35
```

```
echo ages["Alice"]
```

```
for name, age in ages:
```

```
  echo name, " -> ", age
```

ПОРЯДОК В ПРЕДОСТАВЛЕННОМ ЛОГЕ

ages[Alice] = 30

Размер таблицы: 3

Charlie -> 35

Bob -> 25

Alice -> 30

Table не сохраняет порядок вставки. Для него есть OrderedTable.

Deque: очередь с двумя концами

```
p08_collections_advanced.nim · DEQUE[T]

import std/deques

var dq = initDeque[int]()
dq.addLast(1)
dq.addLast(2)
dq.addFirst(0)

echo dq.peekFirst() # 0
echo dq.peekLast()  # 2
```

0 1 2

НАЧАЛО

КОНЕЦ

peekFirst / peekLast

Посмотреть без удаления.

popFirst / popLast

Получить и удалить элемент.

После обоих peek очередь по-прежнему содержит [0, 1, 2].

Демонстрация: один запуск, семь секций

ТЕРМИНАЛ · КАТАЛОГ 02/src

```
nim c -r main.nim
```

Или Docker:

```
docker build -t nim-demo .
```

```
docker run --rm nim-demo
```

main.nim — точка входа

Импортирует отдельные модули и вызывает их run().

Исходники остаются в разных файлах.

01–02 Ошибки

Исключения перехвачены; проверки проходят.

03–04 Файлы и модули

53 байта; файл удалён; square(9) = 81.

05 Обобщения

Два стека и Pair[int, string].

06–07 Коллекции

map/filter, множества, Table и Deque.

Что мы теперь умеем

- 01** Обработать ошибки
try / except / finally и проверяемые утверждения
- 02** Организовывать программу
Модули, экспорт имён и пакеты Nimble
- 03** Повторно использовать код
Generics и собственный Stack[T]
- 04** Выбирать структуры данных
array, seq, set, Table, HashSet и Deque

Далее: конкурентность, парадигмы программирования, VM компилятора и ARC/ORC.

Официальные источники

Язык, исключения, модули

<https://nim-lang.org/docs/manual.html>

Проверки и режимы компиляции

<https://nim-lang.org/docs/assertions.html>

<https://nim-lang.org/docs/nimc.html>

Ввод-вывод и файловые операции

<https://nim-lang.org/docs/syncio.html>

<https://nim-lang.org/docs/os.html>

Generics и семантика копирования

<https://nim-lang.org/docs/tut2.html>

<https://nim-lang.org/docs/destructors.html>

Стандартные коллекции

<https://nim-lang.org/docs/sequtils.html>

<https://nim-lang.org/docs/tables.html>

<https://nim-lang.org/docs/sets.html>

<https://nim-lang.org/docs/deques.html>

Менеджер пакетов Nimble

<https://nim-lang.github.io/nimble/create-packages.html>

<https://nim-lang.github.io/nimble/use-packages.html>